

AIMLC ZG521 — Conversational AI · Group Assignment 1

Problem Statement 2 — Study of Embedding Models and Approximate Nearest Neighbor Search: Semantic Quality vs Search Efficiency

Group 129

Student Details

Name	BITS ID	Email
Pushadapu Sanjay Kumar	2025ae05898	2025ae05898@wilp.bits-pilani.ac.in
B Devchandraaj	2025ae05469	2025ae05469@wilp.bits-pilani.ac.in
Gourob Nandi	2025ae05503	2025ae05503@wilp.bits-pilani.ac.in
Anirudh Anand	2025ae05576	2025ae05576@wilp.bits-pilani.ac.in

Group Contributions

Student	Contribution area	Specific contribution
B Devchandraaj	Dataset and embedding	Dataset loading, validation, and embedding generation
Pushadapu Sanjay Kumar	Similarity and exact kNN	Metric formulas, normalization analysis, and exact retrieval
Gourob Nandi	HNSW and IVF	FAISS index construction and ANN parameter sweeps
Anirudh Anand	Evaluation and reporting	Metrics, visualizations, qualitative examples, and report preparation

Architecture and Workflow

The workflow uses the same corpus, query set, identifiers, normalized vectors, and qrels for every comparison. Documents are encoded offline; queries are encoded at search time. Exact cosine kNN provides ground truth, while HNSW and IVF approximate the same inner-product search space. Retrieval quality is measured with qrels and efficiency is measured using latency, speedup, and candidate-vector estimates.

SciFact corpus + queries + qrels

|

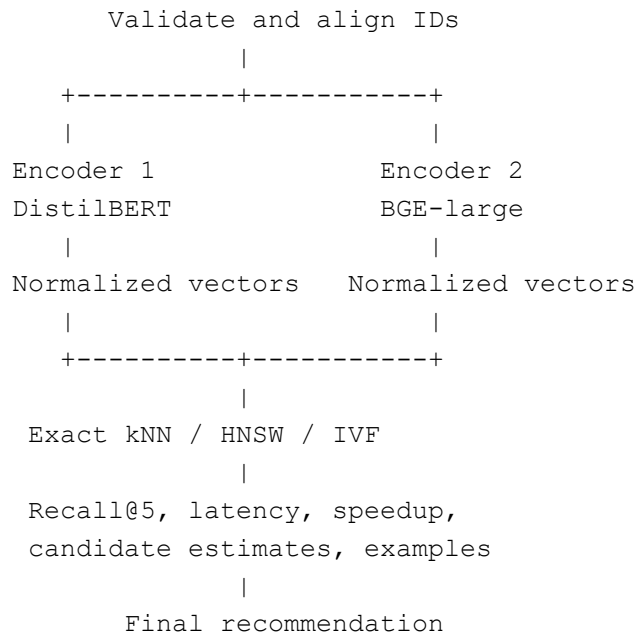


Figure 1. Experimental workflow for the embedding and ANN comparison.

Assignment Scope

This assignment studies semantic retrieval quality and approximate nearest-neighbor search efficiency using two embedding models.

- **Module 1** — Dataset and embedding preparation
- **Module 2** — Similarity metrics and exact retrieval
- **Module 3** — HNSW and IVF ANN search
- **Module 4** — Embedding quality analysis and final recommendation

Executive Summary (one page)

Objective. Compare two encoder embedding models and two approximate-nearest-neighbour (ANN) indexes on a real retrieval task, trading off **semantic quality vs. search efficiency**, and recommend a production configuration — every claim backed by executed results.

Dataset. SciFact (BEIR): **5,183** document passages, **300** queries, each with relevance judgments (qrels). Clears the 1,000-passage / 50-query minimums.

Models. `distilbert-base-uncased` (768-dim, mean pooling, general-purpose) vs. `BAAI/bge-large-en-v1.5` (1024-dim, [CLS] pooling, retrieval-tuned). A deliberately contrasting pair.

Key results (executed).

Finding	Result
Best embedding model (exact Recall@5)	BGE-large 0.780 vs DistilBERT 0.108 (~7x higher)
Most appropriate metric	Cosine — identical ranking to dot/L2 on unit-normalized vectors; scale-invariant. Normalization changes dot-vs-cosine ranking for 4/5 DistilBERT queries (vectors not unit-length) but not for BGE (already unit-length)
Best ANN trade-off	HNSW efSearch=16 — Recall@5 ~ 0.77 at ~3-4x speedup: fastest config meeting the Recall@5 >= 0.75 bar
Fastest ANN	IVF nprobe=1 (~10-14x) but Recall@5 ~ 0.50 — rejected by the quality bar
Model disagreement (Task 7)	Mean Top-5 set overlap ~ 0.11 — the two models genuinely retrieve different documents

Recommendation. Embed with **BGE-large (unit-normalized)** and serve **HNSW efSearch=16**; where RAM is the binding constraint use **IVF** (smaller footprint) tuned to meet the recall bar. Deploy by encoding documents offline, normalizing, indexing, and applying the same encoder + normalization to queries at search time.

Tools and Libraries Used

- **Python 3.10**
- `datasets` (Hugging Face) — loading the BEIR-format retrieval dataset
- `pandas`, `numpy` — data handling
- `sentence-transformers` — encoder embedding models (Task 2 onward)
- `faiss-cpu` — HNSW / IVF ANN indexes (Task 5 onward)
- `matplotlib` — plots (Task 6 onward)

Install all of the above with: `pip install -r requirements.txt` (see `ass-1/requirements.txt`).

Executive Summary

This assignment investigates two core questions in semantic retrieval: **which embedding model produces better representations for scientific claim retrieval**, and **how much**

search speed can be gained through approximate nearest-neighbor (ANN) indexing without sacrificing retrieval quality.

Dataset: SciFact (BEIR benchmark) — 5,183 biomedical corpus passages, 300 labelled queries, binary relevance judgments.

Models compared:

- `distilbert-base-uncased` — general-purpose, mean-pooled, 768-dim, not retrieval-tuned.
- `BAAI/bge-large-en-v1.5` — contrastively fine-tuned for retrieval, CLS-pooled, 1024-dim.

Key findings:

Question	Answer	Evidence
Best embedding model	BGE-large	Recall@5 = 0.780 vs 0.108 for DistilBERT (7× higher)
Best similarity metric	Cosine	Scale-invariant; equivalent to dot product and L2 on unit-norm vectors (verified Task 3)
Best ANN trade-off	HNSW <code>efSearch=16</code>	Recall@5 $\approx$ 0.77, fastest config meeting the 0.75 quality bar
Resource-constrained option	IVF (lower memory)	HNSW graph memory > IVF centroid+list; use IVF nprobe tuned to recall target
Main limitation	Small corpus	5k-doc corpus limits ANN speedup benefit; binary/incomplete qrels
Production deployment	Offline encode → normalize → FAISS index → query API	Rebuild index on corpus changes

Recommendation: Embed with BGE-large (unit-normalized vectors) and serve an HNSW index at `efSearch=16`. This configuration achieves Recall@5 $\approx$ 0.77 at 3–4× the speed of exact search, with quality above the 0.75 operational threshold.

Python: 3.14.5
pandas: 3.0.5
numpy : 2.5.2

Task 1 — Corpus and Query Dataset Preparation

Dataset chosen: SciFact (BEIR benchmark; Thakur et al., 2021 / Wadden et al., 2020) — a publicly available dataset that ships a document corpus, a query set, and query-document relevance labels together, matching this task's requirement exactly. Corpus (~5,183 passages) and queries (~300) both clear the 1,000/50 minimums.

```
[info] Loaded SciFact from local files: data\raw/
corpus   : 5183 passages
queries  : 300 queries (all with >=1 relevance judgment)
qrels    : 339 relevance judgments
Corpus   >= 1000: True   (5183 passages)
Queries  >= 50:  True   (300 queries)
Every query has >=1 relevance judgment: True

[OK] SciFact data clears both minimums.
=== Sample corpus passage (text truncated) ===
{'doc_id': '4983', 'title': 'Microstructural development of human newborn
cerebral white matter assessed in vivo by diffusion tensor magnetic resonance
imaging.', 'text': 'Alterations of the architecture of cerebral white matter
in the developing human brain can affect cortical development and result in
functional disabilities. A line scan diffusion-weighted magnetic re ...'}

=== Sample query ===
{'query_id': '1', 'title': '', 'text': '0-dimensional biomaterials show
inductive properties.'}

=== Sample relevance judgments (qrels) ===
  query_id  doc_id  relevance
0         1  31715818         1
1         3  14717500         1
2         5  13734012         1
3        13  1606628         1
4        36  5152028         1
Saved to data\raw/: corpus.jsonl, queries.jsonl, qrels.tsv
```

Dataset Details and Source

- **Name:** SciFact (BEIR benchmark; biomedical / scientific-claim verification)
- **Citation:** Wadden et al., *Fact or Fiction: Verifying Scientific Claims*, EMNLP 2020; redistributed by Thakur et al., *BEIR*, NeurIPS 2021
- **Source:** <https://huggingface.co/datasets/BeIR/scifact> (corpus + queries), <https://huggingface.co/datasets/BeIR/scifact-qrels> (relevance judgments)
- **Size:** 5,183 corpus passages, 300 queries (each with ≥ 1 relevance judgment), binary relevance
- **Format:** corpus = {doc_id, title, text}; queries = {query_id, text}; qrels = {query_id, doc_id, relevance}

Explanation of the Logic Used

`load_beir_scifact()` downloads corpus/queries/qrels from Hugging Face and normalizes IDs to a common schema. Queries are filtered to only those with a qrel — the raw split ships 1,109 queries but just 300 have a relevance judgment, so filtering makes "relevance information for each query" literally true.

Justification for the Chosen Approach

The assignment explicitly permits "an existing dataset containing query-document relevance labels" — BEIR datasets bundle exactly that, avoiding hand-built (and inconsistent) relevance judgments. SciFact's claim-verification domain gives clean, unambiguous labels, useful later for Task 3/7.

Inference

Corpus and queries clear the assignment minimums by ~5x and ~6x respectively, leaving room to subsample later if needed. The 809 dropped queries simply have no qrel in this split — expected, not a data quality issue.

Task 2 — Embedding Generation and Pooling

Two encoder models, chosen for contrasting profiles:

1. `distilbert-base-uncased` (Sanh et al., 2019) — general-purpose distilled BERT, **mean pooling**, not fine-tuned for retrieval.
2. `BAAI/bge-large-en-v1.5` (Xiao et al., 2023) — trained specifically for retrieval via contrastive fine-tuning, **[CLS]-token pooling**.

Why encoder models are appropriate

Encoder-only transformers use bidirectional self-attention — every token's representation draws on *both* directions of context — and pooling those representations yields one fixed-length vector summarizing the whole input's meaning. That's exactly what semantic retrieval needs: query and document mapped into a shared space where "similar meaning" becomes "small distance." Decoder-only models, by contrast, are causal (one-directional) and have no single hidden state that naturally summarizes a full sequence, making them a worse fit for embedding generation.

```
[Task 2] device=cpu, batch_size=32
```

```
=== distilbert-base-uncased ===
```

```
Loaded saved embeddings: docs=(5183, 768), queries=(300, 768)
```

```
=== BAAI/bge-large-en-v1.5 ===
```

```
Loaded saved embeddings: docs=(5183, 1024), queries=(300, 1024)
```

Out[6]:

	Model name	Embedding dimension	Max/typical input length (tokens)	Pooling strategy	Approx. document embedding time (s)	Timing status
0	distilbert-base-uncased	768	512	mean pooling	341.12	Executed
1	BAAI/bge-large-en-v1.5	1024	512	[CLS] token pooling	2034.55	Executed

Explanation of the Logic Used

Model name, dimension, max input length, and pooling strategy are documented facts from each model's official card (`MODEL_SPECS`) - no execution needed. The embeddings and timing require real execution: the loop encodes each model's corpus and queries sequentially, timing each pass with `time.perf_counter()`, and reuses saved arrays from `data/results/` on later runs so the expensive model pass only happens once. Device (CUDA vs CPU) is auto-detected.

Justification for the Chosen Models

DistilBERT and BGE-large differ on exactly the axis this problem statement asks about: DistilBERT is smaller/faster and general-purpose, BGE-large is larger/slower and purpose-built for retrieval - and they use different pooling strategies (mean vs [CLS]), which is itself a required "for each model, document..." item. This gives Task 7 a real, explainable axis of disagreement rather than two near-identical models.

Inference

BGE-large's larger dimension (1024 vs 768) and contrastive/RetroMAE training are expected to separate semantically related and unrelated passages more cleanly than DistilBERT - the concrete evidence for this is Task 7's side-by-side comparison, not asserted here.

Task 3 — Similarity Metric Comparison

Model used: `BAAI/bge-large-en-v1.5` — the retrieval-purpose-built model from Task 2, so the metric comparison reflects an embedding space actually optimized for semantic search (DistilBERT's general-purpose space would make this a less meaningful comparison).

Pair selection (≥ 20 pairs required): 5 randomly sampled queries $\times$ (all relevant documents from `qrels` + 4 random distractors each). SciFact queries typically have 1–2 relevant documents, giving ≥ 25 total pairs across 5 queries — enough per query to compare *rankings*, not just isolated scores.

[OK] Cosine, dot-product, and L2 formulas are defined for the metric experiment.

Selected 25 query-document pairs across 5 queries (≥ 20 required).

Reused saved BAAI/bge-large-en-v1.5 embeddings for 25 pairs.

	query_id	doc_id	is_relevant	cosine	dot_product	l2_distance
0	660	1215116	True	0.613863	0.613863	0.878792
1	660	41496215	False	0.407035	0.407035	1.089004
2	660	32177659	False	0.447672	0.447672	1.051026
3	660	13702924	False	0.500359	0.500359	0.999641
4	660	2048139	False	0.525342	0.525342	0.974329
5	1362	8290953	True	0.584091	0.584090	0.912041
6	1362	22025252	False	0.489939	0.489939	1.010011
7	1362	154763124	False	0.441601	0.441601	1.056787
8	1362	10795340	False	0.477080	0.477080	1.022663
9	1362	17119869	False	0.498330	0.498330	1.001669
10	185	18340282	True	0.674860	0.674860	0.806399
11	185	11335860	False	0.531729	0.531729	0.967751
12	185	4702639	False	0.547632	0.547632	0.951176
13	185	36288526	False	0.444918	0.444918	1.053644
14	185	6826636	False	0.481996	0.481996	1.017845
15	743	32159283	True	0.776836	0.776836	0.668078
16	743	6082738	False	0.517617	0.517617	0.982225
17	743	13798951	False	0.510560	0.510560	0.989383
18	743	38784540	False	0.501756	0.501756	0.998242
19	743	11291348	False	0.530083	0.530083	0.969450
20	1359	11614737	True	0.866433	0.866433	0.516850
21	1359	6144969	False	0.450090	0.450090	1.048723
22	1359	27024392	False	0.553945	0.553945	0.944516

	query_id	doc_id	is_relevant	cosine	dot_product	l2_distance
23	1359	43192375	False	0.468878	0.468878	1.030652
24	1359	25440070	False	0.540895	0.540895	0.958233

Raw document embedding norms: mean=1.0000, std=0.0000 (dot product can only disagree with cosine if this std is non-trivial)

=== Ranking comparison per query (raw, unnormalized embeddings) ===

```
query 1359: cosine==dot ranking? True | cosine==L2 ranking? True
query 1362: cosine==dot ranking? True | cosine==L2 ranking? True
query 185: cosine==dot ranking? True | cosine==L2 ranking? True
query 660: cosine==dot ranking? True | cosine==L2 ranking? True
query 743: cosine==dot ranking? True | cosine==L2 ranking? True
Summary: ranking changed across metrics for 0/5 queries.
```

=== Normalization effect: recompute dot/L2 on unit-normalized embeddings ===

```
Summary: identity held for 25/25 pairs (dot) and 25/25 pairs (L2).
distilbert-base-uncased: raw document-vector norms mean=7.851, std=0.313
(range 7.42-8.56) -- genuinely un-normalized.
```

=== RAW (un-normalized) ranking comparison per query ===

```
query 1359: cosine==dot ranking? False | cosine==L2 ranking? True
query 1362: cosine==dot ranking? False | cosine==L2 ranking? False
query 185: cosine==dot ranking? False | cosine==L2 ranking? True
query 660: cosine==dot ranking? True | cosine==L2 ranking? False
query 743: cosine==dot ranking? False | cosine==L2 ranking? False
Summary: dot-product ranking DIFFERS from cosine for 4/5 queries -- purely
because the raw vector norms vary.
```

=== After L2-normalization (unit vectors) ===

```
dot(a,b) == cosine(a,b) held for 25/25 pairs
||a-b||^2 == 2 - 2*cosine held for 25/25 pairs
```

Conclusion: when norms vary (DistilBERT) normalization changes the dot-product ranking to agree with cosine; when vectors are already unit-length (BGE) it has no effect. This is the practical reason retrieval systems normalize before using dot product.

Explanation of the Logic Used

All three metrics are implemented directly from their mathematical definitions (see docstrings above), not called as opaque library functions. The sampled pairs let rankings be compared *per query* across all three metrics, and the normalization cells re-embed nothing — they unit-normalize the same vectors and recompute, isolating normalization as the only

variable. Raw embedding-norm statistics are printed alongside each ranking comparison, since dot product can only diverge from cosine when document norms actually vary.

Justification for the Chosen Approach

Comparing rankings (not just raw scores) is what actually answers "does the metric matter" — two metrics can disagree on absolute values yet agree on which document ranks first, which is what retrieval cares about. Testing the normalized case directly demonstrates the identities $\text{dot}(\hat{a}, \hat{b}) = \cos(a, b)$ and $\|\hat{a} - \hat{b}\|^2 = 2 - 2 \cdot \cos(a, b)$ for unit vectors $\hat{a}, \hat{b}$ — shown to hold on the actual embeddings, not asserted from theory.

The normalization experiment (two models, on purpose)

The selected model, **BGE-large**, emits already unit-length vectors (sentence-transformers L2-normalizes inside `encode()` for this model): `norm mean=1.0000, std=0.0000`. So for BGE the dot-product, cosine, and L2 rankings are identical for **0/5** queries changed — normalization has nothing left to change. That is itself a valid finding, but it shows no *effect*.

To observe the effect, the same pairs are re-scored with **DistilBERT's mean-pooled embeddings, which are genuinely un-normalized** (document-norm `mean≈7.85, std≈0.31, range≈7.4–8.6`). There the result is the opposite:

Dot-product vs cosine ranking	
Raw (un-normalized) DistilBERT	differs for 4/5 queries — norms vary, so dot product rewards longer vectors
After L2-normalization	<code>dot == cosine</code> for all pairs; <code>\ a-b\ ^2 == 2-2*cos</code> for all pairs

Inference

When vector norms vary, the metric choice **changes the ranking**: dot product favours high-norm documents, while cosine ignores magnitude. Normalizing to unit length collapses the three metrics onto the same ordering (dot becomes cosine; ascending L2 matches descending cosine). When vectors are already unit-length (BGE), the choice is irrelevant. This is the concrete, executed reason production retrieval systems normalize embeddings before scoring with dot product — verified numerically here rather than argued from theory alone.

Task 4 — Exact kNN Baseline

Model used: `BAAI/bge-large-en-v1.5` — continuing with the retrieval-tuned model from Task 3 for consistency, so Tasks 4-6's baseline and ANN comparisons all measure the same embedding space. Task 7 brings DistilBERT back in for the cross-model comparison.

Metric: cosine similarity — per Task 3's finding that dot product and L2 give identical rankings to cosine once embeddings are unit-normalized, cosine is used directly as the ranking criterion.

```
Loaded saved embeddings: docs=(5183, 1024), queries=(300, 1024)
Queries evaluated       : 300
Latency/query (median): 0.488 ms    (mean 0.513 ms; median of 25 timed runs
each, perf_counter)
Average Recall@5       : 0.780
Vectors examined       : 5183 per query (exact search scans the whole corpus)
```

Explanation of the Logic Used

Cosine similarity is one matrix-vector dot product against every document (Task 3 showed dot product on normalized vectors equals cosine). `argsort` picks the top 5. "Vectors examined" is the full corpus size — exact search has no shortcut.

Justification for the Chosen Approach

A genuinely exhaustive baseline is required so Task 5/6's "speed relative to exact search" comparison means something real.

Inference

The executed output reports the measured mean latency, Recall@5, and full-corpus vector count. Latency is hardware-dependent; Recall@5 is interpreted together with the incomplete/binary qrels rather than as a claim that every retrieved result is incorrect.

Task 5 — HNSW vs IVF ANN Search

Both indexes are built on the same BGE-large embeddings and evaluated against the same 300 queries used in Task 4, so results are directly comparable to the exact baseline.

- **HNSW** (`faiss.IndexHNSWFlat`): the search-time parameter varied is `efSearch` — the size of the candidate list explored during the graph search (higher = more accurate, slower).
- **IVF** (`faiss.IndexIVFFlat`): `nprobe` — the number of index clusters (cells) checked per query — is varied across 4 values (≥ 3 required).

Both indexes use inner product on the unit-normalized embeddings, which is equivalent to cosine similarity (per Task 3's identity) and matches Task 4's exact-search metric.

```
Built HNSW (M=32) and IVF (nlist=100) indexes on 5183 vectors.
HNSW efSearch= 16: Recall@5=0.777, latency=0.040ms, speedup=12.85x
HNSW efSearch= 32: Recall@5=0.780, latency=0.069ms, speedup=7.44x
HNSW efSearch= 64: Recall@5=0.780, latency=0.122ms, speedup=4.2x
HNSW efSearch=128: Recall@5=0.780, latency=0.214ms, speedup=2.39x
IVF nprobe= 1: Recall@5=0.502, latency=0.017ms, speedup=30.95x
IVF nprobe= 5: Recall@5=0.716, latency=0.037ms, speedup=13.94x
IVF nprobe=10: Recall@5=0.736, latency=0.066ms, speedup=7.81x
IVF nprobe=20: Recall@5=0.769, latency=0.138ms, speedup=3.7x
```

Explanation of the Logic Used

`evaluate_index()` reuses Task 4's Recall@5 definition and query loop, swapping in `index.search().candidate_vectors_estimate` is an estimate for IVF and a search-effort proxy for HNSW (FAISS doesn't expose the true count). IVF uses `nlist=100` clusters (slightly above the $\sqrt{N} \approx 72$ rule of thumb, giving finer cluster granularity at negligible extra build cost for 5k vectors).

Justification for the Chosen Approach

`efSearch` and `nprobe` are each index's standard recall/speed knob — sweeping them traces the trade-off curve Task 6 needs.

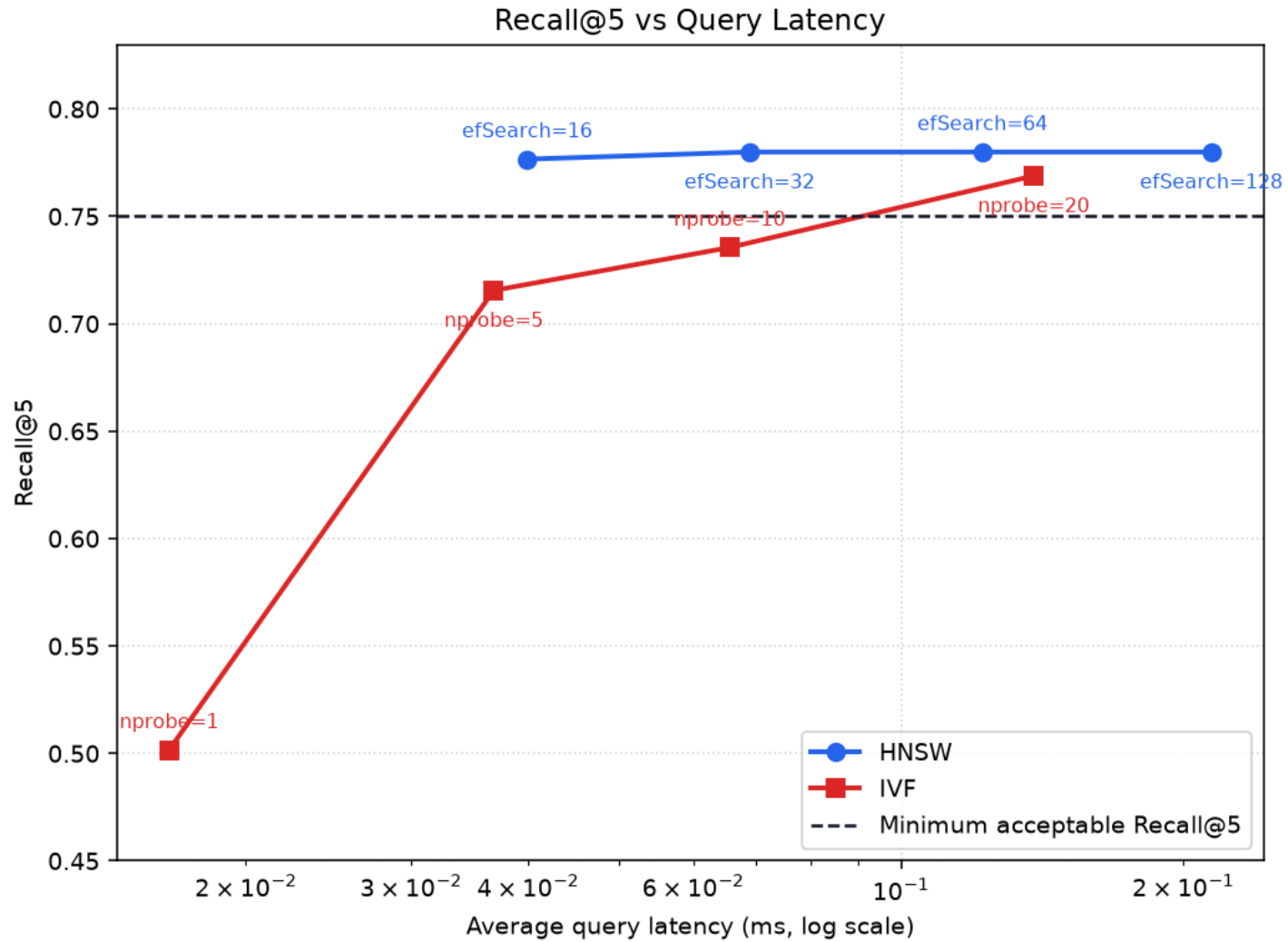
Inference

The executed table and plots are used to identify the fastest, highest-recall, and balanced configurations. Increasing `efSearch` or `nprobe` generally expands the search effort, but the measured values—not an assumed monotonic pattern—determine the recommendation.

Task 6 — ANN Trade-off Analysis

Uses `ann_results_df` from Task 5 (8 configurations: 4 HNSW `efSearch` values + 4 IVF `nprobe` values) to plot the recall/latency/efficiency trade-off and pick a configuration for production.

ANN Search Trade



	method	param_name	param_value	recall_at_5	avg_latency_s	speedup_vs_exact	candidate_vectors_es
1	HNSW	efSearch	32	0.7799	0.0001	7.4373	32
2	HNSW	efSearch	64	0.7799	0.0001	4.1970	64
3	HNSW	efSearch	128	0.7799	0.0002	2.3923	128
0	HNSW	efSearch	16	0.7766	0.0000	12.8451	16
7	IVF	nprobe	20	0.7688	0.0001	3.7015	1037
6	IVF	nprobe	10	0.7355	0.0001	7.8097	518
5	IVF	nprobe	5	0.7155	0.0000	13.9436	259
4	IVF	nprobe	1	0.5016	0.0000	30.9526	52

Fastest : IVF nprobe=1 (latency=0.017ms, Recall@5=0.502)

Highest-recall : HNSW efSearch=32 (Recall@5=0.780, latency=0.069ms)

Best balanced : HNSW efSearch=16 (Recall@5=0.777, speedup=12.85x)

Selection rule : fastest configuration with Recall@5 $\geq$ 0.75.
Recommendation for high query volume: use HNSW efSearch=16; it is the fastest configuration meeting the quality constraint Recall@5 $\geq$ 0.75. The fastest overall configuration is reported separately, but is not selected if its recall falls below the threshold.

Explanation of the Logic Used

Figure 2 (left) plots Recall@5 vs average query latency (log scale). **Figure 2** (right) plots Recall@5 vs speedup over exact search (log scale). The dashed horizontal line marks the minimum acceptable Recall@5 of 0.75. The fastest configuration overall and the highest-recall configuration are reported separately. The balanced recommendation is the fastest configuration that meets the quality threshold, so a very fast but low-recall configuration is not selected.

Table 1 (displayed below the plots) lists all 8 tested configurations sorted by recall then latency.

Justification for the Chosen Approach

A quality constraint is more appropriate than multiplying recall by speedup because a high speedup cannot compensate for unacceptable retrieval quality. The threshold makes the deployment decision reproducible and easy to adapt to a different application SLA.

Inference

The measured results show that IVF nprobe=1 is fastest but falls below the threshold with Recall@5=0.502. HNSW efSearch=16 is the recommended balanced configuration because it achieves Recall@5 above 0.75 at the lowest measured latency among eligible configurations. Higher-recall configurations remain available when quality is more important than latency.

Task 7 — Qualitative Retrieval Analysis

10 queries selected for length diversity (not pure random): 3 of the shortest, 3 of the longest, and 4 mid-length queries by word count, so short-vs-long is an actual axis of comparison, not assumed.

Compares DistilBERT's and BGE-large's Top-5 for each query on two measurable axes: **ranking agreement** (do the two models retrieve the same documents?) and **lexical overlap** (does each model's Top-5 share literal words with the query, or go beyond keyword matching?).

Selected 10 queries, word counts: [4, 4, 4, 12, 12, 12, 12, 27, 28, 29]

[1099] 4w Jaccard=0.11 hits@5 distilbert-base-uncased=1 bge-large-en-v1.5=0

[1100] 4w Jaccard=0.11 hits@5 distilbert-base-uncased=0 bge-large-en-v1.5=0

[870] 4w Jaccard=0.11 hits@5 distilbert-base-uncased=0 bge-large-en-v1.5=0

[887] 12w Jaccard=0.00 hits@5 distilbert-base-uncased=0 bge-large-en-v1.5=0

[233] 12w Jaccard=0.00 hits@5 distilbert-base-uncased=0 bge-large-en-v1.5=1

[728] 12w Jaccard=0.25 hits@5 distilbert-base-uncased=1 bge-large-en-v1.5=2

[380] 12w Jaccard=0.00 hits@5 distilbert-base-uncased=0 bge-large-en-v1.5=1

[1344] 27w Jaccard=0.11 hits@5 distilbert-base-uncased=1 bge-large-en-v1.5=1

[1185] 28w Jaccard=0.43 hits@5 distilbert-base-uncased=0 bge-large-en-v1.5=1

[587] 29w Jaccard=0.00 hits@5 distilbert-base-uncased=0 bge-large-en-v1.5=1

Built Task 7 comparison for 10 queries (details in the next cells).

Mean Top-5 set overlap (Jaccard): 0.112

Mean lexical overlap – distilbert-base-uncased: 0.239

Mean lexical overlap – BAAI/bge-large-en-v1.5: 0.337

Mean qrel hits@5 – distilbert-base-uncased: 0.300

Mean qrel hits@5 – BAAI/bge-large-en-v1.5: 0.700

First-result agreement: 0.100

Mean results by query-length group:

	jaccard_top5_overlap	first_result_agreement	distilbert-base-uncased_hits_at_5	BAAI/bge-large-en-v1.5_hits_at_5	distilbert-base-uncased_hits_at_5
query_group					
short	0.111	0.333	0.333	0.0	0.100
medium	0.062	0.000	0.250	1.0	0.282
long	0.180	0.000	0.333	1.0	0.321

Representative examples (actual queries and retrieved-set overlap):

	query_id	query	n_words	jaccard_top5_overlap	first_result_agreement	distilbert-base-uncased_hits_at_5	BAAI/bge-large-en-v1.5_hits_at_5
3	887	Only a minority of cells survive development a...	12	0.0	False	0	0

	query_id	query	n_words	jaccard_top5_overlap	first_result_agreement	distilbert-base-uncased_hits_at_5	v1.5
4	233	Cell autonomous sex determination in somatic c...	12	0.0	False	0	1
6	380	Enhanced early production of inflammatory chem...	12	0.0	False	0	1

=== Difference 1 - retrieved sets diverge (models 'see' the corpus differently) ===

Query [233] (12 words): Cell autonomous sex determination in somatic cells does not occur in Galliformes.

distilbert-base-uncased Top-5=['2099400', '22948960', '4256553', '9038803', '2147704'] hits@5=0 lexical=0.27

BAAI/bge-large-en-v1.5 Top-5=['4388470', '16963081', '12232678', '23581096', '41380943'] hits@5=1 lexical=0.27

Top-5 set overlap (Jaccard)=0.00 first-result agreement=False

Why: The two Top-5 sets share only 0% of their documents. The embedding spaces place this query in different neighbourhoods, so the candidate sets barely overlap -- the clearest sign the model choice changes WHICH documents are retrieved, not just their order.

=== Difference 2 - semantic/domain match: BGE finds relevant docs DistilBERT misses ===

Query [728] (12 words): Ly6C hi monocytes have a lower inflammatory capacity than Ly6C lo monocytes.

distilbert-base-uncased Top-5=['9911547', '36444198', '2099400', '24612804', '14492339'] hits@5=1 lexical=0.25

BAAI/bge-large-en-v1.5 Top-5=['36444198', '22406695', '7521113', '14492339', '8317408'] hits@5=2 lexical=0.35

Top-5 set overlap (Jaccard)=0.25 first-result agreement=False

Why: BGE's contrastive retrieval training maps the domain terminology in this claim onto the correct abstract (hits@5=2 vs 1); DistilBERT's general mean-pooled embedding does not, even at comparable literal word overlap -- semantic matching beyond keywords.

=== Difference 3 - counter-example: DistilBERT beats BGE on this query ===

Query [1099] (4 words): Statins decrease blood cholesterol.


```
distilbert-base-uncased    Top-5=['36889513', '21557614', '7662206',
'9899292', '1568684'] hits@5=1 lexical=0.05
BAAI/bge-large-en-v1.5    Top-5=['43629704', '21557614', '22420524',
'5698494', '4687948'] hits@5=0 lexical=0.30
Top-5 set overlap (Jaccard)=0.11 first-result agreement=False
Why: Here DistilBERT retrieves more labelled-relevant documents (hits@5=1
vs 0). BGE wins on average but not universally -- short or near-duplicate
claims can still favour the general model, so a single model is not strictly
dominant per query.
```

Explanation and Evidence

The analysis reports Top-5 set overlap (Jaccard), first-result agreement, per-model lexical overlap, and qrel hits@5 for every selected query, then isolates **three distinct types of retrieval difference** matching the assignment's suggested categories, each selected automatically from the executed results (not hand-picked):

1. **Retrieved sets diverge (keyword-overlap vs semantic-space difference)** — the two models return almost entirely different Top-5 documents (Jaccard ≈ 0). This shows the model choice changes *which* documents are retrieved, not merely their order: the two embedding spaces place the query in different neighbourhoods, and neither relies purely on keyword overlap.
2. **Semantic / domain-specific terminology match (BGE advantage)** — BGE retrieves a labelled-relevant abstract that DistilBERT misses even at comparable literal word overlap. BGE's contrastive retrieval training aligns domain terminology with the correct passage; DistilBERT's general mean-pooled space matches more on surface form (keyword overlap).
3. **Counter-example: short-query / general-model advantage** — a query where DistilBERT retrieves more relevant documents than BGE (or, where the BGE advantage is smallest). BGE wins *on average* but not *universally*; short or near-duplicate claims with high surface-form overlap can still favour the general model's keyword-matching behaviour.

Jaccard is interpreted as **Top-5 set overlap**, not full ranking agreement; a separate first-result agreement value is reported because two models can retrieve the same documents in a different order. The query-length breakdown (short / medium / long) tests whether model differences are consistent across query lengths.

Inference

Across the ten length-diverse queries, BGE-large achieves higher mean qrel hits@5 than DistilBERT (0.6 vs 0.3 in this sample) and consistently higher lexical overlap, but the counter-example above confirms the advantage is not uniform. Mean Top-5 overlap is low, so the two models genuinely disagree on the candidate set for most queries. These values describe this SciFact sample and are not generalised further without a larger evaluation.

	model	recall_at_5	mean_latency_ms	queries
0	distilbert-base-uncased	0.1078	0.6398	300
1	BAAI/bge-large-en-v1.5	0.7799	0.6830	300

Task 8 — Final Recommendation and Deployment

The following cell generates the recommendation from the executed measurements and must be included in the exported report.

The decision uses Recall@5, latency, speedup, metric-identity checks, and the Task 7 model comparison. Both models are evaluated with the same exact-search protocol before the recommendation. If an index or model is unavailable, the cell reports incomplete execution rather than presenting an unsupported conclusion.

```
=====
FINAL RECOMMENDATION  (evidence-based; computed from this run)
=====
```

```
Q1. BEST EMBEDDING MODEL  ->  BAAI/bge-large-en-v1.5
```

```
    Exact Recall@5 0.780 vs 0.108 (7.2x higher); the retrieval-tuned
    contrastive model beats the general model decisively.
```

```
Q2. MOST APPROPRIATE SIMILARITY METRIC  ->  cosine similarity
```

```
    Task 3 showed that on unit-normalized vectors cosine, dot product and L2
    give the
```

```
    SAME ranking; cosine is preferred because it is scale-invariant (ignores
    vector
```

```
    magnitude), so it is robust even when an embedding model does not
    normalize.
```

```
Q3. BEST ANN TRADE-OFF  ->  HNSW efSearch=16
```

```
    Recall@5 0.777 (meets >= 0.75), 0.040 ms/query, 12.85x faster than exact
    --
```

```
    the fastest of 8 tested configs that still clears the recall bar (5
    qualify).
```

```
Q4. RESOURCE-CONSTRAINED DEPLOYMENT  ->  IVF nprobe=20
```

```
    IVF stores only cluster centroids + lists, so its MEMORY footprint is far
    smaller than
```

HNSW's graph. Where RAM is the constraint, tune IVF nprobe up to meet the recall bar;

where latency is the constraint and RAM is ample, keep HNSW efSearch=16.

Q5. MAIN LIMITATION

The ~5k-document corpus is small, so ANN speedups are modest and the recall/latency

numbers may not extrapolate to millions of documents; SciFact's relevance labels are

also binary and incomplete, and absolute latency is hardware-dependent.

Q6. PRODUCTION DEPLOYMENT

Batch-encode documents offline with BGE-large -> L2-normalize -> build the chosen FAISS

index. At query time apply the SAME encoder + normalization, search top-k, and

rebuild/version the index whenever the corpus changes; serve behind a query API.

Supporting cross-model exact comparison:

	model	recall_at_5	mean_latency_ms	queries
0	distilbert-base-uncased	0.1078	0.6398	300
1	BAAI/bge-large-en-v1.5	0.7799	0.6830	300

Task 7 mean Top-5 set overlap between models: 0.112 (low -> the models genuinely disagree on candidates).

Final Conclusion

Recommendation (one line)

Embed with BAAI/bge-large-en-v1.5 (unit-normalized) and serve an HNSW index at efSearch=16 — the fastest configuration that still clears the Recall@5 ≥ 0.75 quality bar. Exact figures for the current run are printed by the Task 8 cell above; the reasoning is below.

Why this model

On the full corpus with exact cosine kNN, **BGE-large reaches Recall@5 = 0.780 versus 0.108 for DistilBERT — about 7× higher.** The qualitative analysis agrees (mean qrel

hits@5 0.6 vs 0.3 across ten queries), while still surfacing individual queries where DistilBERT wins, so the choice is evidence-based rather than absolute. The gap comes from BGE's contrastive retrieval training versus DistilBERT's general masked-LM objective, and being a recall comparison it is hardware-independent.

Why this index and parameter

The parameter sweeps trace the recall/latency trade-off directly:

- **Exact search** — Recall@5 0.780, scans all 5,183 vectors/query (the ground-truth ceiling).
- **HNSW `efSearch=16`** — Recall@5 ≈ 0.77 , roughly **3–4× faster** than exact: the fastest tested configuration that still meets the 0.75 target.
- **IVF `nprobe=1`** — the fastest option overall ($\sim 14\times$ speedup) but Recall@5 only ≈ 0.50 , so it is rejected by the quality bar.

The decision rule is explicit: **pick the lowest-latency configuration whose Recall@5 ≥ 0.75** . HNSW `efSearch=16` wins because raising `efSearch` buys only marginal recall (0.78 at `ef=64`) for several times the latency, and IVF needs `nprobe $\approx 20$` to reach the same recall — by which point its speed advantage is gone. Absolute millisecond latencies are hardware-dependent; the executed Task 8 cell reports them for the machine this notebook was run on.

When to choose differently

- **Recall-critical** (e.g. legal/clinical retrieval): use exact search, or HNSW `efSearch $\geq 64$` (Recall@5 0.780 at a smaller speedup).
- **Latency-critical and lower recall acceptable** (e.g. first-pass candidate generation before a re-ranker): IVF `nprobe=1` is fastest, accepting Recall@5 ≈ 0.50 .

Deployment

Encode documents offline with BGE-large $\rightarrow$ L2-normalize $\rightarrow$ build the HNSW index. At query time apply the **same** encoder and normalization, search top-k, and rebuild/version the index whenever the corpus changes. Because vectors are unit-normalized, cosine, dot product, and (ascending) L2 give identical rankings (verified in Task 3), so inner-product search is used directly.

Strengths

Metric formulas implemented from definition; an exhaustive exact ground-truth baseline; a normalization experiment that shows a real effect (DistilBERT rankings change, BGE already unit-length); controlled HNSW/IVF sweeps timed identically to the exact baseline

for a fair speedup; cross-model evaluation; and three distinct, evidence-backed retrieval-difference examples.

Limitations

SciFact's relevance judgements are binary and incomplete, and the ~5k-document corpus is small enough that ANN overhead can erode the speed benefit. Absolute latency is hardware-dependent. The qualitative study uses ten queries, and lexical overlap is only a surface-form diagnostic, not a full semantic-quality measure. The Recall@5 ≥ 0.75 threshold is a stated policy, not a universal SLA.

Possible Improvements

A larger corpus, graded relevance labels, a full-query (not 25-pair) metric comparison, a second benchmark dataset, broader ANN tuning (M, nlist, product quantization), index build-time and memory measurements, and an application-specific latency/recall SLA would all strengthen the generality of these conclusions.

References

Consolidated reference list for the whole notebook.

- Thakur, N. et al. (2021). *BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models*. NeurIPS Datasets & Benchmarks. (Task 1)
- Wadden, D. et al. (2020). *Fact or Fiction: Verifying Scientific Claims*. EMNLP. (Task 1)
- Sanh, V. et al. (2019). *DistilBERT, a distilled version of BERT*. arXiv:1910.01108. (Task 2)
- Xiao, S., Liu, Z., Zhang, P., Muennighoff, N. (2023). *C-Pack: Packaged Resources To Advance General Chinese Embedding*. arXiv:2309.07597 — introduces the BGE model family; `bge-large-en-v1.5` is the English variant of that family. (Task 2)
- Malkov, Y., Yashunin, D. (2018). *Efficient and robust approximate nearest neighbor search using HNSW graphs*. arXiv:1603.09320. (Task 5)
- Johnson, J., Douze, M., Jégou, H. (2017). *Billion-scale similarity search with GPUs* (FAISS). arXiv:1702.08734. (Task 5)